This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2889662, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

Estimate and Recompute: A Novel Paradigm for Approximate Computing on Data Flow Graphs

Mingze Gao and Gang Qu
{*mgao1, gangqu*}*@umd.edu*
Department of Electrical and Computer Engineering and Institute of Systems Research
University of Maryland, College Park, MD, USA

***Abstract*—Approximate computing is an emerging computation paradigm that utilizes many applications' intrinsic error resilience to improve power and energy efficiency. Several approaches have been proposed to identify the non-critical computations by analyzing the output sensitivity to the accuracy of the results, and then perform approximation on these computations. However, these static approaches only use the prior knowledge (e.g. input ranges) for analysis and fail to consider the runtime information, which limits the energy saving and incurs large computation error. In this paper, we propose a runtime approximate computing framework to solve this problem. The basic idea is to use a low cost method to estimate the impact of each immediate input value to the accuracy of computation at every node in the data flow graph (DFG), and then decide whether we should simply use the estimated value or perform an accurate computation. Our novel runtime estimation method is based on converting data to the logarithmic representation. We propose two algorithms to make the decision at certain nodes whether an accurate computation will be needed to balance energy saving and computation error. Compared to the static DFG node cutting approach, our approach's estimation accuracy is 32x better to achieve the same amount of energy saving. Furthermore, we combine our dynamic design methodology with the traditional static approach and propose a new algorithm to improve the runtime efficiency. We apply this algorithm to two machine learning algorithms, kmeans and perceptron. The results show that we can save up to 57.22% computations with only 0.27% error in kmeans and 91.41% with 0.72% error in perceptron.**

***Index Terms*—Energy efficient, data flow graph, approximate computing, logarithmic estimation, runtime**

I. Introduction

NOWADAYS, energy efficiency has become one of the most crucial concerns in modern computing system design. Meanwhile, a wide range of applications, such as multimedia/signal processing, data mining, and machine learning, tolerate the non-accurate computations as long as the overall quality is acceptable. Approximate Computing (AC) is a design methodology that takes advantage of these intrinsic error-resistant features of applications and produces "good enough" result to enhance the energy efficiency. The existing state-of-the-art designs of approximate computing can be classified into levels of software, compiler, architecture, memory, and circuit [14]. In this paper, we focus on approximating the data flow graph by hardware and software co-design.

A data flow graph (DFG) is a directed graph which represents the data dependencies and flow directions between a number of operations. Data flow graph has multiple variants that do not need to be in the real shape of a directed graph. Abstract Syntax Tree (AST) which is used in [5], can be converted to DFG easily. Besides, a slice of software program can also be treated as a DFG where the nodes are the computational operations and the edges are the corresponding variables or intermediate results. Therefore, our proposed approach is transplantable in any other software program variants.

In this paper, we utilize the error resilient characteristics of the operations (e.g. add/sub, min/max) in the data flow graph to identify the non-critical nodes. For example, $S = add(A, B)$ where both A and B are intermediate results coming from previous computations, if the absolute value of B is much smaller than A's, the computational error of S will be less related, in other words, less sensitive to B. Therefore, instead of computing accurately, we can replace B with an estimated value. Similarly, for max/min operation, we can completely neglect the computation of one operand as long as we are able to determine it is smaller/larger than the other one.

[5], [6], [11] share very similar design philosophy with ours. The energy savings can be achieved by approximating the non-critical parts of the program. These non-critical parts are targeted by using training data or by data range tuning and interval arithmetic. However, all these methods are working in offline regardless of every input, as we called "static". To guarantee the accuracy in the worst case, this static feature brings the *"under-approximate"* issue. For example, a part of the program is sensitive to the final output in general, but for some given inputs it becomes insensitive and negligible. Since in [5], [6], [11], the computations to be approximated is predecided, they cannot foresee these special cases.

Our proposed approach will dynamically consider each input data during execution, and decide whether a node is critical for that input. The major challenge of this "dynamic" fashion is how to estimate the intermediate values accurately and energy efficiently. To address this challenge, we propose a novel estimation approach and the corresponding highly hardware-efficient computing mechanism. Before the estimation process, input data is converted into fixed point base-2 logarithmic representation. During the estimation, the operations in the data flow graph are transformed into the computations in logarithmic domain. We use the estimated result to decide the non-critical parts of the graph. Then we approximate the DFG by replacing these non-critical parts with the estimated values. Our experiments demonstrate that the logarithmic estimation process will be much more energy efficient compared with accurate computing in linear domain,

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2889662, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

leveraging the overall energy efficiency enhancement of the system. Part of our work has been published in [20].

II. Related work

[1] has done the detailed summary of existing approximate computing strategies and techniques. The author reviewed the existing research in both hardware and software, as well as programming frameworks. In this section, we discuss the work in two fields that related to our approach, the arithmetic, and the software program analysis.

In approximate arithmetic design field, the power or energy savings is achieved either by designing the state-of-the-art arithmetic units or shrinking the bitwidth of the operands. Ye et al. [4] proposed a re-configurable approximate adder which allows for adjustment of accuracy with trade-off with power consumption. A scalable multiplier with unbiased dynamic range is introduced in [3]. It dynamically truncates the least significant bits (LSBs) to save energy. [13] proposed a framework of JPEG compression that allows the user to specify the approximate bitwidth, and dynamically adjust the approximation level based on the input images. [9] proposed an Approximate Integer Format (AIF) and its corresponding mechanism that can be used for all arithmetic operations. They dynamically select the most significant bits (MSBs) into computations. AIF effectively shortens the bitwidth of the operands within guaranteed error rate. Furthermore, [16], [17] extend approximate arithmetic into security field to balance the tradeoff between energy efficiency and security demand for IoT devices. Besides the arithmetic units, [15] proposed an approximate lookup table based accelerator which stores and yields approximate result of given function without computation. [7], [8] studied both non-iterative and iterative approximate logarithmic multipliers (ALMS) to reduce the power consumption and improve performance. The rationale of their design is very similar to our estimation process. Compared with the above work, our approach mainly focuses on the data flow level approximation rather than customizing the arithmetic units. The arithmetic operations during proposed estimation process are only used for the non-critical branches, and the overall accuracy is dominated by the critical branches. Therefore, we pay more attention to optimize the arithmetic cost, such as energy/power and timing, instead of the accuracy during estimation.

[5] proposed a novel automated approximation technique for the logic synthesis. They first transform the exact RTL or behavioral HDL design into an Abstract Syntax Tree (AST). Then they applied training data to locate the approximate parts in the AST and replace them with their approximate variants. Finally, they wrote back the modified AST to RTL or behavioral HDL. In [6], a novel sensitivity computation approach named Automatic Sensitivity Analysis (ASAC) is proposed to extract the sensitivity of the output with respect to the given program data by tuning the data range of variables. Then the non-critical data can be isolated based on the achieved sensitivity information. The energy saving is leveraged by computing and storing these non-critical data approximately. Similarly, interval arithmetic (IA) is a numerical method to compute the bounds of given variables efficiently. But it may suffer from the wrapping effect which outputs largely overestimated bounds. [11] proposed an algorithmic differentiation based interval computing approach that can effectively reduce the wrapping effect in many cases. The authors adopt this IA variant to analyze the significance of variables in the program with respect to the final output. The insignificant part of the program can be either executed in low precision or unreliable hardware or simply replaced by constants. These approaches could bring the *under-approximate* issue as illustrated in the fourth paragraph in the introduction.

Moreover, the above three approaches can also cause the *"over-approximate"* issue. [5] uses training data to determine where and how to approximate. Whether the training data is sufficient will directly affect the quality of approximation. Insufficient training data may misclassify an important part to be non-critical, leading to over-approximation issue. [6], [11] use input ranges to tune the approximation level. If the input ranges are not strictly bounded, some uncovered inputs may cause significant errors. Khudia et al. in [12] propose a computationally inexpensive framework named Rumba, for online detection and large approximation error correction. Rumba offers the continuous lightweight checks and fixes these errors by recomputing exactly to solve the *"over-approximate"* problem. But it cannot capture the *"under-approximate"* cases. In comparison, our approach considers each input runtimely. Therefore, both *"under/over-approximate"* issues can be well addressed.

Some other works have been reported targeting the approximate computing on DFG. For example, [10] well addressed the approximate resource allocation and binding problem. They proposed a novel error propagation model and scheduling algorithms to maximize the energy saving within the given accuracy constraints.

III. Arithmetic estimation in Logarithmic domain

A. Conversion from Floating Point to Logarithmic

In this paper, we target on the floating point operations due to its ubiquitous usage. There are two main types of floating point format: single precision and double precision. Double precision is widely used for those applications with precision concern, such as in the scientific computing field. But those precision driven applications are usually not the candidates of the approximate computing due to energy efficiency concerns. In our work, we focus on approximating the single precision computation. There are 32 bits in IEEE754 single precision standard consisting of 1 sign bit, 8 exponent bits, and 23 mantissa bits, as shown in the Fig.1 (a). The value of a single precision floating point number can be computed by

$$sign \times (1 + mantissa) \times 2^{exponent-127} \tag{1}$$

By ignoring the sign bit, we can rewrite eq (1) for a given x as $x = 1.m \times 2^e$, where the dot is the radix point. Let x_l to be the logarithmic value of x. The subscript l denotes the logarithmic representation, and we use this notation for both

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2889662, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

data and transformed arithmetic operations in the rest of the paper. Then x_l can be approximated as

$$\begin{aligned} x_l &= \log_2(x) = \log_2(1.m \times 2^e) \\ &= \log_2(1.m) + e \quad (2) \\ &\approx m + e = e.m \quad (3) \end{aligned}$$

From eq (2) to (3) we use the approximation equation:

$$\log_2(1+x) \approx x \qquad \text{for } x \in [0,1] \quad (4)$$

Based on the derived result in (3), we can get x_l by directly truncating the floating format without any computation (shown in Fig.1(b)). In eq (4), for $x \in (0,1)$, $\log_2(1+x)$ is always slightly larger than x. To minimize this error, we empirically truncate mantissa at the 5th bits without rounding the 6th. x_l is represented as a fixed point with a sign bit in front. The radix point is at the same position with the floating point format. When recovering the data from log domain to linear, we can simply pad "0"s to the end and change the data type from fixed point to floating point. The conversion and recovery process are shown in Fig. 1.

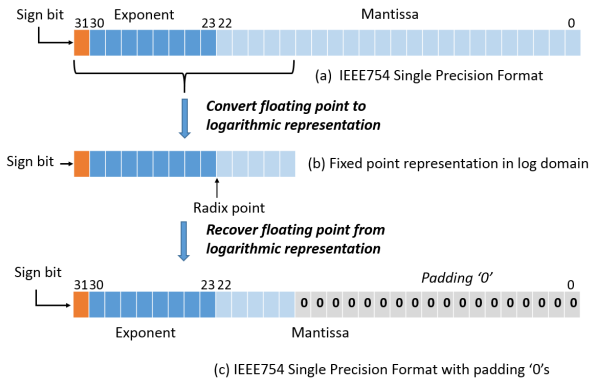


Fig. 1. Conversion of floating point formation between linear domain and log domain

B. Arithmetic Operations in Logarithmic

The above subsection illustrated how to convert the input data into log domain. Next, we discuss how to use this converted data in the basic arithmetics. Let A and B are two input operands in linear domain, and S is the arithmetic output. The converted operands are A_l and B_l, and the output in log domain is S_l. For simplicity, we assume A and B are positive.

1) Accurate Conversion: The following operations can be directly processed without adding error compensation.

1 Multiplication: $S = A \times B$
$S_l = \log_2 S = \log_2(A \times B) = \log_2 A + \log_2 B = A_l + B_l$
Multiplication in log domain, marked as mul_l can be done by simply adding the logarithmic operands.

2 Divison: $S = A/B$
Similar with multiplication, $S_l = A_l - B_l$

3 Square root: $S = \sqrt{A}$
$S_l = \log_2 S = \log_2 \sqrt{A} = \frac{1}{2}\log_2 A = \frac{1}{2}A_l = A_l \gg 1$

4 Power n: $S = A^n$
$S_l = \log_2 A^n = n\log_2 A = nA_l$

5 Min/Max or comparison: $S = max(A, B)$
$S_l = \log_2 S = \log_2 max(A,B) = max(\log_2 A, \log_2 B) = max(A_l, B_l)$
Comparative operations stay the same with linear domain.

The key character in the above operations is that: the computation processes are much cheaper in log domain than those in linear domain. For example, the multiplication and division of floating point unit (FPU) are very energy expensive and time consuming. But in log domain, we can simply use a fixed point adder with 13 bitwidth. Compared with single precision floating point multiplier, this 13-bit fixed adder can achieve more than 100x energy saving. Similarly, computing square root is intricate for computer, but in log domain we only need a shift register.

2) Approximate Conversion: However, addition and subtraction are much more complicated in log domain. Intuitively, we first need to the recovery step in subsection A to convert the data back to linear domain. Then apply FPU for addition and subtraction, and redo the conversion step to get the logarithmic representation of the result. However, our goal in log domain is to classify the non-critical subgraph through estimation, therefore we do not need the result to be very accurate as long as we can tell which operations are negligible. To this point, we introduce a novel addition/subtraction estimation with error compensation technique. Without loss of generality, we assume A and B are positive and $A > B$:

$$\begin{aligned} S &= A + B = 2^{A_l} + 2^{B_l} \\ &= 2^{A_l}(1 + 2^{B_l - A_l}) \\ &\approx 2^{A_l}(1 + 2^{round(B_l - A_l)}) \\ &= 2^{A_l}(1 + 2^{-ed}) \quad (5) \end{aligned}$$

Where the exponential difference $ed = round(A_l - B_l)$, and $round()$ means round to nearest integer. Since $A_l > B_l$, ed will be a positive integer, and $2^{-ed} \in (0,1)$. Therefore

$$\begin{aligned} S_l &= \log_2 S = \log_2 2^{A_l}(1 + 2^{-ed}) \\ &= \log_2 2^{A_l} + \log_2(1 + 2^{-ed}) \approx A_l + 2^{-ed} \\ &= \begin{cases} A_l + 1 \gg ed & \text{if } ed \le 5 \\ A_l & \text{if } ed > 5 \end{cases} \end{aligned}$$

Here we reuse the eq(4). Notice that A_l has only 5 bits in fraction part. So we will omit the compensation if $ed > 5$.

We draw a motivational example in Fig. 2 to illustrate the above derivation. Clearly, we can see that our new proposed logarithmic addition can be estimated by only using two fixed point additions and one shift.

The estimation process of subtraction is similar with addition at the beginning but has different compensation.

$$\begin{aligned} S &= 2^{A_l} - 2^{B_l} \approx 2^{A_l}(1 - 2^{-ed}) \\ S_l &= \log_2 S = A_l + \log_2(1 - 2^{-ed}) \quad (6) \end{aligned}$$

In above, we cannot apply eq (4) for approximation. Instead, we use Look-up Table to store the approximate value of $\log_2(1 - 2^{-ed})$. The values are shown in Table.I. Similar with

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2889662, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

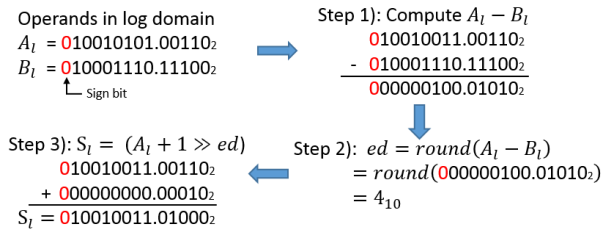


Fig. 2. Example of addition in log domain

addition, we will discard the error compensation if $ed > 5$. A special case is when $ed = 0$. It means B_l is just slightly smaller than A_l. But in linear domain, the possible value of $A - B$ could vary within $(0, 0.293 * A)$. Eq (6) cannot handle this special case because $\log_2(0)$ is $-\infty$, which is infeasible in practice. In the table, we empirically set the compensation value to -1 for easy computation. Based on the experiments we observe that this value has very minor effect on the overall estimation quality.

TABLE I
LOOK UP TABLE FOR SUBTRACTION ESTIMATION IN LOG DOMAIN

ed	$\log_2(1-2^{-ed})$	binarized approximation	binarized value
0	~	-1.00000_2	-1
1	-1	-1.00000_2	-1
2	-0.415	-0.01110_2	-0.4375
3	-0.1926	-0.00110_2	-0.1875
4	-0.0931	-0.00011_2	-0.09375
5	-0.0458	-0.00001_2	-0.03125

We can also construct the Look-up table for $\log_2(1+2^{-ed})$ in addition estimation. But in general, shift operation with only a few steps is cheaper than checking the Look-up Table. Besides, the difference between these two ways of error compensations is negligible based on our experiment.

[19] uses base-2 logarithmic representation to encode the weights in convolutional neural network. It looks like that their approach is very similar to our logarithmic estimation approach. But in fact, they are fundamentally different. [19] quantizes $\log_2(x)$ to integer and apply shifting operations during accumulating. In our approach, we use fixed point number to represent $\log_2(x)$, and we only need adder and comparator during the whole estimation process. Apparently, our approach provides much higher accuracy and [19] fails to guarantee the computational quality of traditional applications such as DSP.

IV. RUNTIME DFG APPROXIMATION

Section III introduced how to estimate the intermediate value by transforming the data flow graph into logarithmic domain. In this section, we explain how to use these estimated data to replace the non-critical parts of the DFG. For simplicity, we only consider the directed acyclic graph (DAG). In the real world applications, a DFG will possibly have loops. If the number of loops is deterministic, we can unroll the loop to make it acyclic. If the number of loops is non-deterministic, we can focus on one single cycle that is acyclic. Besides, we can apply dependency analysis and add these loop-sensitive nodes into whitelist, and always apply accurate computations for the nodes in the whitelist.

A. Non-criticality Truncation

1) Error Resilient and Sensitive Operations: The operations inside a DFG with more than two operands can be decomposed into multiple operations that have only one or two inputs. Given a decomposed DFG, we classify these operations into two types, the error sensitive operations and error resilient operations. Each node in DFG denotes an operation, and each edge can be considered as an intermediate result (except the input and output edges).

Definition 1.1: The node in a DFG with error sensitive operation is defined as an error sensitive node, denoted as n^s.

The nodes with operations like multiplication, division or exponentiation are considered as error sensitive nodes since any tiny variation on their inputs could bring significant difference on the output.

Definition 1.2: The node in a DFG with error resilient operation is defined as the error resilient node, denoted as n^r. An n^r is dominated by one input, named as I_d, and is less sensitive to the other minor input, marked as I_m.

For $S = add(A, B)$ in linear domain, the dominant input I_d is the one with larger absolute value of A and B, and I_m is the rest one. For $S_l = add_l(A_l, B_l)$, the dominant input I_d equals to $max(A_l, B_l)$. In log domain, we no longer need to take the absolute value of each inputs. Because the signs of A_l,and B_l also reflect magnitude of their original values in linear domain. Similarly, I_d will be the one with smaller input for $S = min(A, B)$ and $S_l = min_l(A_l, B_l)$ since the output will completely not be affected by the larger one.

We explore this error resilient feature to identify the non-critical inputs and their corresponding branches.

2) Non-criticality Definition and Classification: The non-critical input only occurs when the node is error resilient. All the inputs of error sensitive nodes will be treated as critical inputs. In this paper, we consider the error resilient node with the addition/subtraction and comparative (such as min/max) operations.

Definition 2: An input I_m is a non-critical input in log domain if and only if $f(I_d - I_m) \geq \delta$, where

$$f(x) = \begin{cases} x & \text{for } add_l/sub_l \\ abs(x) & \text{for comparative operations} \end{cases}$$

where δ is the threshold we use to control the overall computational quality. $I_d - I_m = 1$ means that the dominant input is twice larger than the minor input in the linear domain, regardless their signs. The larger δ is, the less critical I_m will be. For the comparative operations, we compute the absolute difference between two inputs. This is because I_d could be either the larger one or the smaller one depending on the comparison type. The criticality is determined after the estimation process.

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2889662, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

3) Truncation and Recomputation: The following steps list how to produce the approximate result for energy efficiency after targeting the non-critical input of the given error resilient node n_i^r.

1 Cut off the n_i^r's parent branch that computing the non-critical input;
2 Replace non-critical input with the estimated value I_m;
3 Convert the estimated value I_m in log domain to the value in linear domain;
4 Recompute the n_i^r's critical parent branch in linear domain accurately;
5 Compute node n_i^r accurately.

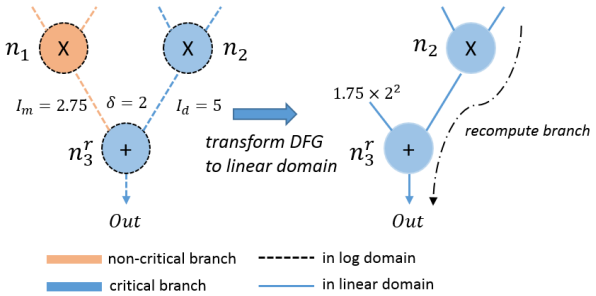


Fig. 3. Example of truncation and recomputation

In step 1, all the nodes in the non-critical parent branch are removable no matter it is error resilient or not. Fig .2 presents a motivation example of this truncation and recomputation step.

4) Error Analysis: In this section, we deduce a theoretical error analysis of the truncation and recomputation steps. We consider addition as an example. Let v_m and v_d are the two accurate input values of n_i^r with I_m and I_d as their logarithmic values. Therefore $v_m = 2^{I_m}$ and $v_d = 2^{I_d}$. Assume the estimation error of I_m and I_d are ε_m and ε_d. So the recovered value of v_m is $2^{I_m+\varepsilon_m}$. The reason we use absolute error ε_m instead of an error rate is that the experimental result shows that the value of ε_m and I_m are highly independent. For simplicity, we omit the conversion and recovery error between the log and linear domain. So the error rate of node n_i^r is:

$$\begin{aligned} er &= \frac{O_{apx} - O_{acc}}{O_{acc}} = \frac{(2^{I_m+\varepsilon_m} + v_d) - (v_m + v_d)}{v_m + v_d} \\ &= \frac{2^{I_m+\varepsilon_m} - v_m}{v_m + v_d} = \frac{2^{I_m+\varepsilon_m} - 2^{I_m}}{2^{I_m} + 2^{I_d}} \\ &\leq \frac{2^{\varepsilon_m} - 1}{2^{\delta-\varepsilon_d+\varepsilon_m} + 1} \end{aligned} \tag{7}$$

in eq (7), we use the relation

$$(I_d + \varepsilon_d) - (I_m + \varepsilon_m) \geq \delta$$

O_{acc} and O_{apx} are the accurate and approximate output respectively. Assume $\varepsilon_m - \varepsilon_d \approx 0$, then

$$er \leq \frac{2^{\varepsilon_d} - 1}{2^{\delta} + 1} \tag{8}$$

If we let $\varepsilon_m = 0.2$, which means we have around 15% estimation error, and $\delta = 2$ and bring them into eq (8):

$$er \leq \frac{2^{0.2} - 1}{2^2 + 1} \approx 2.97\%$$

With a properly selected threshold, we can balance the tradeoff between the computational quality and energy saving.

B. Runtime Approximation Algorithms

We are now able to determine whether the parent branches of a given node n_i^r is non-critical or not, but we cannot directly truncate all the non-critical nodes we found in the whole data flow graph. Consider the following scenario:

Problem 1: A parent node n_k is in the non-critical branch of n_i^r, but it is also in the critical branch of n_j^r. Simply removing n_k may not cause significant error of n_i^r, but it will have a bad impact on n_j^r.

To solve this problem, we propose two efficient graph truncation algorithms, which can be integrated with the runtime scheduler of data flow graph.

1) GlobalCut Algorithm: The *GlobalCut* algorithm provides an global consideration of DFG. Instead of maximally removing the non-critical nodes, we translate the problem to: minimally reserve the critical nodes on the path from the primary inputs to the primary outputs.

Algorithm 1: *GlobalCut* Algorithm

```
Input  : A data flow graph G; an input set I_i
Output: Executable nodes list Exec
estimate( G, I_i);
Exec = ∅ ;
for each output node O_i do
    Q = queue(O_i); path_i = ∅ ;
    while Q! = ∅ do
        currNode = Q.pop() ;
        path_i.add(currNode) ;
        if currNode is n^s then
            Q.push(currNode.parents) ;
        else if I_m is non-critical then
            Q.push(node_{I_d}) ;
        else
            Q.push(node_{I_d}, node_{I_m}) ;
    Exec = Exec ∪ path_i
```

In the algorithm, we start from the output nodes and go backward to the inputs, and find out the critical path of each output. $estimate(G, I_i)$ is to estimate the graph in log domain under input I_i. Q is a queue initialized with i_{th} output node O_i. $path_i$ contains all the critical nodes of O_i. $node_{I_d}$ and $node_{I_m}$ are the parent nodes of $currNode$ outputting I_d and I_m. The algorithm is running in $O(V+E)$, where V and E are the numbers of nodes and edges. Normally in DFG, we have $E \approx V$. So the time complexity of *GlobalCut* grows linearly with the number of nodes in the graph. *GlobalCut* returns the executable nodes remaining in the graph. Approximate output can be obtained by simply executing the nodes in $Exec$ topologically in linear domain.

2) LocalCut Algorithm: The *GlobalCut* will estimate the whole graph using a given input. But the estimated value will not update when we iteratively execute the nodes in $Exec$.

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2889662, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

Therefore, the errors of I_m and I_d increase with the depth of the graph going up, and propagate to the final output. To solve this drawback, we propose a *LocalCut* algorithm. The detailed pseudocode is listed below. $n_i.parents.val_l$ means the logarithmic value of n_i's parents nodes.

Algorithm 2: $LocalCut$ Algorithm

```
Input  : A data flow graph G;
         An input set I_i
Output: The final result of G
for  n_i in G's topological order do
    if n_i is n^s then
        estimate(n_i, n_i.parents.val_l);
    else if n_i is n^r then
        if I_m is non-critical then
            recompute I_d branch in linear domain;
        else
            recompute I_d, I_m branch in linear domain;
    Compute n_i in linear domain;
    Convert n_i to log domain for next estimation ;
```

Different with *GlobalCut*, the *LocalCut* goes forward. It will:

1 Iteratively estimate the graph till meeting the next error resilient node n_i^r.
2 Do the truncation and recomputation steps illustrated in subsection 3) for n_i^r.
3 Go back to 1 using n_i^r's recomputed result.

Similar with *GlobalCut*, the timing complexity of *LocalCut* is also $O(V)$. G's topological order can be precomputed since it is not related to the input values. The *LocalCut* algorithm solves the problem 1 implicitly. If n_k is non-critical to n_i^r, *LocalCut* will replace it by estimated value. While if n_k at the same time is critical to n_j^r, the algorithm will recompute it. But the output might be affected by the execution order of n_i^r and n_j^r. If we compute n_j^r first, the result of n_k is accurate. Then the next time we compute n_i^r, we can directly use the accurate value of n_k instead of the estimated value. But on the other hand, if n_i^r comes before n_j^r, the algorithm will adopt n_k's estimated value to compute n_i^r. The *LocalCut* will be not able to determine the optimal order of n_i^r and n_j^r with lack of "global" concern. So this algorithm is greedy.

The *LocalCut* algorithm always executes the error resilient nodes. So typically its energy saving is less than the *GlobalCut*. However, the paths from the primary inputs to primary outputs are split by the error resilient nodes in *LocalCut*, and each subpath is usually much shorter than those in *GlobalCut*, therefore effectively reducing the estimation error propagation.

C. Conditional Approximation Algorithm

The above two algorithms can effectively cut the non-critical nodes for every input during runtime. However, this feature relies on the runtime decision of the branch's criticality. What's worse, the *GlobalCut* and *LocalCut* needs V_r decisions in the worst cases, where V_r is the number of error resilient nodes. These large number of the decisions will severely affect the efficiency of the algorithms.

To reduce the number of decisions, we propose a static and runtime hybrid algorithm. The static feature can be used to reduce the decision overhead, and the runtime feature guarantees the accuracy. We apply a training procedure for the given data flow graph. We run the *GlobalCut* algorithm T times (T=100k in this paper), and count the times of each node being cut. Then we select the most frequently cut nodes and reshape them into C fanout cones. In the runtime, instead of making a potentially large amount of decisions, we only need to determine whether these C fanout nodes is critical or not to their children nodes, and recompute the corresponding cones if critical. We name this new method as *ConditionalCut*.

Definition 3: Let cnt_i denote the times of node i being cut during T runs. $P_i = cnt_i/T$ denotes the probability of cutting node i, and S_i denotes the energy saving of node i if being cut. The expectation of energy saving will be $E = \sum P_i S_i$.

Lemma 1: If node k is only in node i's cone and node i is the fanout node, then: $P_k = P_i$.

Lemma 2: If node k is in the intersection of cone i and cone j, then: $P_k = P_i + P_j - P(cone_i \bigcap cone_j)$, where "$\bigcap$" means $cone_i$ and $cone_j$ will be cut at the same time.

The *ConditionalCut* can be considered as a problem that: select the nodes to formulate into C cones in order to maximize E. The optimal solution is very difficult to find because: 1) Intuitively we want to add more nodes into cones to achieve more savings, but adding the nodes may change the probabilities of its predecessors (Lemma 1). 2) It is very difficult to calculate the exact probabilities of the intersection nodes (Lemma 2). This is because $cone_i$ and $cone_j$ may or may not be independent due to the graph's shape or even the specific value of the input. To compute the probability of a single intersection node exactly, we need to store all the nodes that were cut in each time amongst T times in total, and count the conditional appearances. What's worse, we need to count again and again when $cone_i$ and $cone_j$ changes. And things become more complicated for the nodes in the intersection of more than 2 cones. The huge memory and timing costs make this not feasible. To solve this problem, we propose a heuristic approach.

We notice a fact: $cnt_i = \sum_{j \in node_i.children} cnt_j$. This means that the parents always have larger cnt than their children, and if we greedily adding nodes in the descending order of cnt, the nodes will eventually connect consecutively from the primary inputs towards the primary outputs. Besides, instead of maximizing E directly, we can evaluate the candidate solution t times and select the one that has the maximum energy saving. The training part of the algorithm is shown in Alg.3.

In this algorithm, line 1-3 count the cut frequencies of each node with the complexity of $O(TV)$. Then starting from line 5, we iteratively add the most frequently cut node to the candidate pool (apxNodes). Line 8-10 find the fanout nodes amongst the pool and store them into condNodes. Line 9 can be done in $O(1)$ in the real coding, so the complexity of line 8-9 is $O(V)$. Line 11-13 collect the solutions that the number of conditional nodes is no more than C. Line 14-23 generate the

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2889662, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

Algorithm 3: $ConditionalCut$ Algorithm $--$ Training

Input : A data flow graph G; Number of conditions C
Output: A list of candidates contains: Conditionally approximate nodes $condNodes$; Accurate nodes $accNodes$

```
1  run GlobalCut T times;
2  for each node n_i in G do
3      count the times of being cut Cut_{n_i};
4  apxNodes = ∅, candidates = [] ;
5  for n_i in the desending order of Cut_{n_i} do
6      apxNodes.add(n_i), condNodes = ∅ ;
7      for n_i in apxNodes do
8          if n_j.children ∩ apxNodes ≠ n_j.children then
9              condNodes.add(n_j);
10     if size(condNodes) ≤ C then
11         accNodes = getAccNodes(condNodes) ;
12         candidates.append([condNodes, accNodes]) ;
13 Function getAccNodes(condNodes)
14     accNodes = [] ;
15     for each output node O_i do
16         Q = queue(O_i) ;
17         while Q ≠ ∅ do
18             currNode = Q.pop() ;
19             if currNode not in condNodes then
20                 Q.push(currNode.parents) ;
21                 accNodes.append(currNode) ;
22     return accNodes;
```

node that always needs to be accurately computed with the complexity of $O(V)$. The overall complexity of the training part is $O(TV + V^2)$.

After we get the candidates of accNodes and condNodes from training step, we run each candidate t times and obtain averaged energy savings and select the one with the maximum saving. The pseudocode is shown in the runtime part.

Algorithm 4: $ConditionalCut$ Algorithm $--$ Runtime

Input : A data flow graph G; an input set I_i; condNodes and accNodes from Training part;
Output: The final result of G

```
1  estimate( condNodes, I_i);
2  for node n_i in the reverse order of accNodes do
3      if a parent p is a conditional node then
4          if p is critical to p.children then
5              recompute p's branch in linear domain ;
6          else
7              use p's estimated value;
8      compute n_i
9  return each O_i
```

In line 2, we traverse the node in the reverse order of accNodes. This is because the reversed accNodes is inherently sorted in topology order. The "if" statement in line 3 can be predecided after training. The complexity of each run is $O(V)$, so the overall evaluation procedure takes $O(tV)$. Unlike training, we do not need t to be very large, as long as we can figure out the best combinations.

The *ConditionalCut* algorithm can also be implemented in programming level if the logarithmic operation introduced in section III has hardware and instructions support. Consider a piece of code that computes $a * b + c + d$. If we can observe that $a*b+c$ is frequently cut after training, we can reformulate the code to be

```
float dfg(float a, float b, float c,
          float d){
    logint apx = logadd(logmul(a,b), c);
    if (apx + threshold <= logint(d)){
        return float(apx)+d;
    }else{
        return a*b+c+d;
    }
}
```

In the example, the logadd and logmul are the arithmetic operations shown in section III.B, logint is the logarithmic representation format of a float number. The logint() and float() are the conversion and recovery steps in Fig.1. The if condition judges whether the branch $a * b + c$ is critical to d or not.

D. Estimation Timing Overhead Elimination

The estimation process could bring additional timing overhead if the estimated branch will be recomputed. In this subsection, we heuristically provide two techniques for the proposed algorithms to eliminate this timing overhead.

1. Schedule the estimation process in pipeline

This technique is suitable for *GlobalCut* and *ConditionalCut* algorithm. Given a series of input $I_0, I_1, ...I_n$ that will be executed in order, in the normal way, when running the GlobalCut for input I_1, we first need to $estimate(G, I_0)$, then decide which nodes will be cut off and which nodes will be executed accurately. When I_2 comes, we need to do the same step again. The $estimate()$ brings additional timing overhead. However, if we utilize the fact that the $estimate()$ and accurate computations do not have hardware dependency since they require fixed point and floating point operations individually, we can schedule these two processes in parallel. For example, when executing I_1, we can call the $estimate(G, I_2)$ concurrently and determine which nodes will be cut off for I_2. Then after I_1 is done, we can directly execute the I_2 without estimation. This approach does not work for *LocalCut* because the estimation is processed simultaneously with the computation in *LocalCut*.

2. Parallelize estimation and computation

For *LocalCut*, we can estimate the other nodes when executing the undeletable nodes.

Definition 4: A node n_k is undeletable if it can reach both inputs of an error resilient node n_i^r, and there is no other error resilient node lying in the path from n_k to n_i^r.

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2889662, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

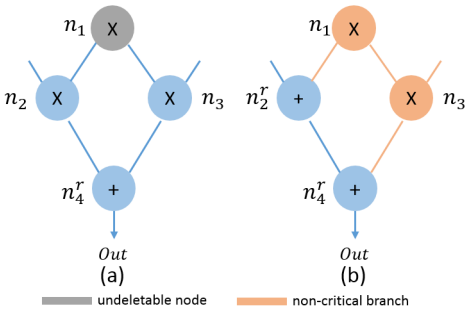


Fig. 4. Examples of undeletable node

An example is shown in Fig.4. In DFG (a), node n_1 will always be executed no matter which parent branch of n_4^r is cut off. So n_1 is an undeletable nodes in DFG (a). But it is not the case in DFG (b). The branch from n_1 to n_4^r might be broken by other error resilient node n_2^r, if n_1 lies on its non-critical branch and n_3 is also non-critical to n_4^r. The undeletable node can be found before the execution as long as we know the topological order of the nodes. *GlobalCut* algorithm cannot use this technique because even n_i^r itself might be cut off if it lies on the non-critical path of another error resilient node.

E. Integrating with Approximate Arithmetic

In the traditional approximate arithmetic design field, researchers are focusing on designing the approximate arithmetic units. Most of these designs are incompatible with each other and can only be used individually. However, our proposed approaches can be easily integrated with the existing approximate arithmetic designs. This is because we only truncate the graph, but do not have specific limitations on how to execute the rest nodes during the recomputation process. Therefore, we can adopt the approximate units for the untruncated nodes to achieve additional power savings. Besides, if I_m is an estimated value, pursuing the extremely high accuracy of I_d becomes less worthy. In this paper, we use a straightforward way to compute the floating point operation approximately. We use only 10 mantissa bits during the computation. Different from the estimation process, where we truncate 5 mantissa bits and convert data from linear domain to log domains, these 10 bits will perform the same computation logic with the original 23 bits but with much shorter bitwidth.

V. EXPERIMENTAL RESULT

In this section, we first compare the power consumption of some basic arithmetic operations between the linear and log domain. To evaluate the computational qualities and energy savings, we implement *GlobalCut* and *LocalCut* algorithms and test them on the DFG benchmarks [2]. Then we use 10 mantissa bits as an approximate arithmetic technique for these two algorithms to see the influence of errors as well as the energy savings. Moreover, to emphasize the advantages of the runtime feature in our approach, we compared the error rate of our algorithms with a naive static-based *FixCut* approach at the same energy savings. We also compare the hybrid algorithm *ConditionalCut* with previous two purely runtime algorithms. In the end, we use two machine learning examples to show that the *ConditionalCut* could be extremely powerful.

A. Arithmetic Operation Power Comparison

We implement the floating point units and our proposed logarithmic arithmetic estimation blocks in Verilog and synthesize them using Cadence RTL Compiler with FreePDK 45nm library. Clearly, we can see from Table IV that the power consumptions of arithmetic in log domain are 40x $\sim$ 150x smaller than those in linear domain. The significant saving can be obtained if we use estimated value instead of computing accurately.

B. Accuracy/Energy Saving vs Threshold

We slightly modify the data flow graph from [2]. For each DFG, we randomly generate 100,000 inputs. We implement our two proposed algorithms with threshold δ from 1 to 4 to approximate these graphs based on each input. We report the averaged mean absolute error and energy savings in Table II. From the result, we observe that:

1 Larger threshold reduces both errors and energy savings.

2 In most cases, *GlobalCut* has more energy savings compared with *LocalCut*. This is due to the fact that *LocalCut* always executes the error resilient nodes.

3 *LocalCut* usually outputs lower error rate than *GlobalCut*. Like we discussed in section IV.A, this is because all the error resilient nodes will be accurately computed in the *LocalCut* algorithm, therefore largely limits the error magnitude during propagation.

TABLE II
ACCURACY AND ENERGY SAVINGS OF *GlobalCut* AND *LocalCut* UNDER DIFFERENT THRESHOLD

Applications	$\delta=1$				$\delta=2$				$\delta=3$				$\delta=4$			
	GlobalCut		*LocalCut*		*GlobalCut*		*LocalCut*		*GlobalCut*		*LocalCut*		*GlobalCut*		*LocalCut*	
	error	**saving**	**error**	**saving**	**error**	**saving**	**error**	**saving**	**error**	**saving**	**error**	**saving**	**error**	**saving**	**error**	**saving**
hal	6.71%	59.06%	6.73%	59.00%	1.64%	49.68%	1.66%	49.39%	0.74%	38.47%	0.77%	38.25%	0.31%	27.88%	0.33%	27.67%
cosine1	8.49%	26.73%	2.80%	16.95%	3.95%	14.99%	0.87%	10.66%	2.21%	8.00%	0.34%	6.42%	1.18%	4.39%	0.12%	3.83%
cosine2	10.17%	28.03%	4.29%	17.19%	4.93%	16.15%	1.57%	10.97%	2.19%	8.65%	0.51%	6.65%	0.91%	4.70%	0.09%	3.94%
fir1	3.29%	52.58%	2.45%	35.97%	1.74%	36.29%	1.47%	27.47%	0.78%	22.96%	0.73%	19.49%	0.30%	13.75%	0.31%	13.05%
fir2	3.43%	51.98%	3.26%	32.35%	1.66%	34.98%	1.66%	22.74%	0.56%	19.75%	0.58%	13.30%	0.16%	10.49%	0.17%	7.14%
autoRegFilter	2.79%	41.36%	1.43%	21.58%	0.89%	21.33%	0.46%	12.65%	0.24%	10.17%	0.13%	7.07%	0.06%	4.97%	0.04%	3.94%
downsample	3.04%	49.36%	2.24%	4.61%	1.39%	32.51%	0.97%	4.61%	0.26%	19.51%	0.17%	4.61%	0.05%	17.58%	0.04%	4.61%
ellipticWaveFilter	0.86%	13.42%	0.76%	5.14%	0.20%	5.71%	0.16%	2.63%	0.04%	3.04%	0.03%	1.40%	0.01%	2.23%	0.01%	0.81%
average	4.85%	40.31%	2.99%	24.10%	2.05%	26.46%	1.10%	17.64%	0.88%	16.32%	0.41%	12.15%	0.37%	10.75%	0.14%	8.12%

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2889662, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

TABLE III
ACCURACY AND ENERGY SAVINGS OF $GlobalCut$ AND $LocalCut$ WITH ONLY 10 MANTISSA BITS

Applications	$\delta=1$				$\delta=2$				$\delta=3$				$\delta=4$			
	$GlobalCut$		$LocalCut$		$GlobalCut$		$LocalCut$		$GlobalCut$		$LocalCut$		$GlobalCut$		$LocalCut$	
	error	**saving**	**error**	**saving**	**error**	**saving**	**error**	**saving**	**error**	**saving**	**error**	**saving**	**error**	**saving**	**error**	**saving**
hal	6.7%	76.9%	6.72%	77.17%	1.65%	72.99%	1.67%	73.26%	0.76%	68.31%	0.78%	68.71%	0.34%	63.85%	0.36%	64.37%
cosine1	8.99%	61.28%	2.83%	55.97%	4.15%	55.53%	1.01%	53.42%	2.38%	52.12%	0.52%	51.68%	1.37%	50.18%	0.31%	50.59%
cosine2	10.29%	62.0%	3.87%	56.06%	5.0%	56.14%	1.28%	53.55%	2.3%	52.46%	0.67%	51.78%	1.12%	50.39%	0.31%	50.63%
fir1	3.47%	73.81%	2.4%	66.52%	1.73%	66.52%	1.43%	63.09%	0.72%	60.83%	0.69%	59.86%	0.26%	56.99%	0.27%	57.25%
fir2	3.74%	72.71%	3.22%	61.46%	1.77%	64.3%	1.62%	57.58%	0.57%	56.92%	0.55%	53.76%	0.16%	52.31%	0.16%	51.22%
autoRF	3.25%	70.65%	1.39%	61.58%	1.01%	61.64%	0.43%	57.96%	0.27%	56.67%	0.12%	55.67%	0.08%	54.21%	0.05%	54.32%
downsampling	4.43%	70.04%	2.07%	43.6%	1.93%	58.78%	0.86%	43.6%	0.32%	51.85%	0.17%	43.6%	0.12%	50.74%	0.11%	43.6%
ellipticWaveFilter	1.22%	51.47%	0.71%	47.76%	0.3%	47.15%	0.17%	46.69%	0.09%	45.32%	0.06%	46.16%	0.05%	44.63%	0.05%	45.9%
average	5.26%	67.36%	2.90%	58.76%	2.19%	60.38%	1.06%	56.14%	0.93%	55.56%	0.45%	53.90%	0.44%	52.91%	0.20%	52.24%

TABLE IV
POWER COMPARISON BETWEEN LOG AND LINEAR DOMAIN

Power Consumption (mW)	**add**	**sub**	**mul**	**div**
Log domain	1.0	1.2	0.53	0.53
Linear domain	43.2	43.2	71.2	51.9

4 Different benchmark has different performance. For example, $ellipticWaveFilter$ has very low errors but its energy savings are much less than the others. This is because the nodes in this benchmark have highly interfered and most of the nodes cannot be easily removed without affecting others.

C. Integrating with Approximate Arithmetic

As mentioned in section IV.E, we apply only 10 bits in mantissa during the recomputation process in both algorithms. The results are listed in Table. III. Compared with Table.II, we can see that: fewer mantissa bits will slightly increase the errors but can achieve about 45% additional savings. The 45% is estimated by:

$$\frac{Table.IV\ saving - Table.III\ saving}{1 - Table.III\ saving}$$

One may argue that using the fewer mantissa seems to bring more energy savings than proposed algorithms, which devalues our contribution. But in fact, our methods are still well worth to be implemented because: 1) We focus on truncating the non-critical part of the graph using very low energy cost. The saving ratios shown in Table.III can be approximately treated to be the portions of the graphs being cut. 2) Our approach does not rely on any specific approximate arithmetic units, which means that no matter which approximate arithmetic units will be used, our approach can always contribute additional savings by reducing the number of computations. For example, reducing the mantissa bits has very limited room for the half precision floating point system, which has 5 bits in exponents and 10 bits in mantissa. But our approach is still applicable.

D. Comparison with FixCut

We randomly generate 100,000 inputs as training data, and apply *GlobalCut* algorithm with $\delta = 2$ for each input. We permanently replace the nodes that are most frequently removed in *GlobalCut* with their averaged values under training inputs. We name this static approximation as *FixCut*. Table V lists the errors when running *FixCut* on training data and new data. For comparison, we make the energy savings of *FixCut* to be the same with *GlobalCut*. Obviously, the lack of input oriented concerns causes the tremendous errors. The data in column *GlobalCut* is directly copied from Table II for comparison convenience.

TABLE V
$FixCut$ ERROR COMPARED WITH $GlobalCut$

benchmarks	**GlobalCut**	**new data**	**training data**
hal	1.64%	109.72%	111.25%
cosine1	3.95%	72.53%	82.85%
cosine2	4.93%	257.99%	231.44%
fir1	1.74%	15.89%	15.41%
fir2	1.66%	16.52%	16.35%
autoRegFilter	0.89%	13.68%	13.68%
downsample	1.39%	18.75%	18.62%
ellipticWaveFilter	0.20%	18.12%	17.85%
average	2.05%	65.40%	63.43%

E. Comparison with ConditionalCut

We force the $\delta = 2$, and use 100k inputs to train the *ConditionalCut* with the number of conditions varying from 3 to 9. The errors and energy savings are listed in Table.VI. The results of *GlobalCut* and *LocalCut* are directly copied from Table.II for comparison convenience. C denotes the number of conditions, as well as the decisions. We can see that:

1 Increasing C gives more energy savings but larger errors. (For *hal* benchmark, the results remain the same when $C = 5, 7, 9$. This is because the number of conditions saturated after $C = 5$.)

2 The results of *ConditionalCut* are usually between those of *GlobalCut* and *LocalCut*.

3 More importantly, the majority of the savings can be achieved with only 5 or 7 conditions, which is far less than those of the other two algorithms. Therefore, it can be a good compromise between the savings and the runtime efficiency of the algorithm.

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2889662, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

TABLE VI
ACCURACY AND ENERGY SAVINGS OF $conditionalCut$ WITH DIFFERENT NUMBER OF CONDITIONS

Applications	$GlobalCut$			$LocalCut$			$C=3$		$C=5$		$C=7$		$C=9$	
$\delta=2$	**error**	**saving**	**C**	**error**	**saving**	**C**	**error**	**saving**	**error**	**saving**	**error**	**saving**	**error**	**saving**
hal	1.64%	49.68%	4.96	1.66%	49.39%	5	1.21%	50.81%	1.34%	50.82%	1.34%	50.82%	1.34%	50.82%
cosine1	3.95%	14.99%	23.4	0.87%	10.66%	26	3.45%	6.24%	2.9%	6.93%	3.01%	7.64%	3.32%	8.35%
cosine2	4.93%	16.15%	23.0	1.57%	10.97%	26	1.7%	5.46%	2.51%	7.05%	2.71%	8.01%	2.83%	8.83%
fir1	1.74%	36.29%	9.22	1.47%	27.47%	10	1.02%	15.87%	1.25%	21.2%	1.42%	26.58%	1.51%	29.36%
fir2	1.66%	34.98%	11.20	1.66%	22.74%	15	1.25%	21.0%	1.51%	28.55%	1.61%	31.03%	1.66%	32.31%
autoRF	0.89%	21.33%	10.9	0.46%	12.65%	12	0.17%	6.03%	0.6%	14.56%	0.74%	17.42%	0.81%	18.81%
downsampling	1.39%	32.51%	20.2	0.97%	4.61%	30	0.69%	22.54%	1.1%	24.13%	1.34%	26.79%	1.51%	28.39%
ellipticWaveFilter	0.20%	5.71%	25.12	0.16%	2.63%	26	0.16%	5.16%	0.19%	5.8%	0.2%	6.34%	0.22%	6.74%
average	2.05%	26.46%	16	1.10%	17.64%	18.75	1.21%	16.64%	1.42%	19.88%	1.54%	21.82%	1.65%	22.94%

F. ConditionalCut in Machine Learning Applications

The benchmarks we use have massive additions and subtractions. Based on the theoretical error analysis in section IV A.4, these two operations will propagate the errors no matter how large the threshold is. But for comparative operations like min/max, it is possible to get the accurate result with a properly selected threshold. These comparative operations are more error resilient than additions and subtractions, leveraging significant energy savings with very minor computation quality loss. In this section, we apply our *ConditionalCut* algorithm on two well-known applications, kmeans and perceptron.

The goal of perceptron is to compute the label y, where $y = sign(W^T x + b)$ (assume linear). W and b are trained weights and bias, and x is the input data. The branch $W^T x + b$ will be cut as long as we know its sign. Therefore, we borrow the idea of *ConditionalCut* and heuristically insert two conditions, "$\geq \delta$" or "$\leq -\delta$" right after $W^T x + b$. We estimate the sign using the approach in section III, and only recompute y for those whose estimated value of $W^T x + b$ is within $(-\delta, \delta)$.

In the kmeans application, a point will be clustered to the nearest center. So we need to compute the distances between a given point to all the centers, and find the minimum value amount these distances. We use our approach to quickly estimate the distances, and recompute those that are no larger than the minimum estimated distance plus threshold.

These two applications share the similar design concept with [18], where the authors proposed the Iterative CNN (ICNN), which reformulates the CNN to a series of smaller networks, and the networks terminate when reaching acceptable classification confidence.

TABLE VII
PREDICTION ERROR AND ITERATION SAVINGS USING CONDITIONALCUT

	$\delta=1$		$\delta=2$	
Applications	**#iter saved**	**error**	**#iter saved**	**error**
Kmeans	57.22%	0.27%	42.33%	0.0%
Perceptron	91.41%	0.72%	79.34%	0.12%

The error of kmeans is the percentage of the number of points that are mis-clustered. We report the number of iterations saved when computing the distances between the points and centers in Table VII. For perceptron, the error means the percentage of misclassified data compared with accurate computing. We test the pre-trained perceptron classifier with 10,000 randomly generated data. By setting $\delta = 1$, 91.41% of data can be correctly classified by only estimating y instead of accurate computing with only 0.72% error rate.

VI. CONCLUSION

In this paper, we proposed a novel runtime estimation technique that can energy-efficiently estimate the results of the arithmetic operations without accurate computation. Based on this estimation technique, we proposed three effective data flow graph truncation algorithms. The algorithms can dynamically truncate the non-critical parts of given data flow graph based on the estimated value with the concern of every input. These approximations effectively leverage considerable energy savings with acceptable computation quality. Besides, we showed that our algorithm could be very powerful for some machine learning algorithms.

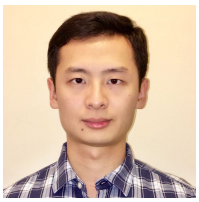
REFERENCES

[1] Sparsh Mittal. 2016. "A Survey of Techniques for Approximate Computing". ACM Comput. Surv. 48, 4, Article 62 (March 2016)
[2] http://www.ece.ucsb.edu/EXPRESS/benchmark/
[3] S. Hashemi, R. I. Bahar and S. Reda, "DRUM: A Dynamic Range Unbiased Multiplier for approximate applications," 2015 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), Austin, TX, 2015, pp. 418-425. doi: 10.1109/ICCAD.2015.7372600
[4] Rong Ye, Ting Wang, Feng Yuan, Rakesh Kumar, and Qiang Xu. 2013." On reconfiguration-oriented approximate adder design and its application". In Proceedings of the International Conference on Computer-Aided Design (ICCAD '13). IEEE Press, Piscataway, NJ, USA, 48-54.
[5] Kumud Nepal, Yueting Li, R. Iris Bahar, and Sherief Reda. 2014. "ABACUS: a technique for automated behavioral synthesis of approximate computing circuits". In Proceedings of the conference on Design, Automation & Test in Europe (DATE '14).
[6] Pooja Roy, Rajarshi Ray, Chundong Wang, and Weng Fai Wong. 2014. "ASAC: automatic sensitivity analysis for approximate computing". SIGPLAN Not. 49, 5 (June 2014), 95-104.
[7] Weiqiang Liu, Jiahua Xu, Danye Wang, and Fabrizio Lombardi. 2017. Design of Approximate Logarithmic Multipliers. In Proceedings of the on Great Lakes Symposium on VLSI 2017 (GLSVLSI '17). ACM, New York, NY, USA
[8] W. Liu, J. Xu, D. Wang, C. Wang, P. Montuschi and F. Lombardi, "Design and Evaluation of Approximate Logarithmic Multipliers for Low Power Error-Tolerant Applications," in IEEE Transactions on Circuits and Systems I: Regular Papers.
[9] M. Gao, Q. Wang, A. S. K. Nagendra and G. Qu, "A novel data format for approximate arithmetic computing", 2017 22nd Asia and South Pacific Design Automation Conference (ASP-DAC), Chiba, 2017, pp. 390-395.
[10] Chaofan Li, Wei Luo, S. S. Sapatnekar and Jiang Hu, "Joint precision optimization and high level synthesis for approximate computing", 2015 52nd ACM/EDAC/IEEE Design Automation Conference (DAC), San Francisco, CA, 2015, pp. 1-6.
[11] J. Riehme and U. Naumann "Significance analysis for numerical models" 1st Workshop on Approximate Computing (WAPCO) 2015

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2889662, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

[12] D. S. Khudia, B. Zamirai, M. Samadi and S. Mahlke, "Rumba: An online quality management system for approximate computing", 2015 ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA), Portland, OR, 2015, pp. 554-566.
[13] F. S. Snigdha, D. Sengupta, J. Hu and S. S. Sapatnekar, "Dynamic Approximation of JPEG Hardware," in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems.
[14] Qiang Xu, Nam Sung Kim, and Todd Mytkowicz."Approximate Computing: A Survey". In: vol. 33. 1.Feb. 2016, pp. 822.
[15] Y. Tian, T. Wang, Q. Zhang and Q. Xu, "ApproxLUT: A novel approximate lookup table-based accelerator," 2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), Irvine, CA, 2017, pp. 438-443.
[16] M. Gao, Q. Wang, Md T. Arafin, Y. Lyu, G. Qu: "Approximate Computing for Low Power and Security in the Internet of Things". IEEE Computer 50(6): 27-34 (2017)
[17] M. Gao and G. Qu, "A novel approximate computing based security primitive for the Internet of Things," 2017 IEEE International Symposium on Circuits and Systems (ISCAS), Baltimore, MD, 2017
[18] Neshatpour, F. Behnia, H. Homayoun and A. Sasan, "ICNN: An iterative implementation of convolutional neural networks to enable energy and computational complexity aware dynamic approximation," 2018 Design, Automation & Test in Europe Conference & Exhibition (DATE), Dresden, 2018, pp. 551-556.
[19] Daisuke Miyashita, Edward H. Lee and Boris Murmann: "Convolutional Neural Networks using Logarithmic Data Representation". In CoRR. abs/1603.01025. 2016, http://arxiv.org/abs/1603.01025
[20] M. Gao and G. Qu, "Energy efficient runtime approximate computing on data flow graphs," 2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD), Irvine, CA, 2017



Mingze Gao received his B.S in Electronic Science and Technology in 2012 from Wuhan University, Hubei, China, and Ph.D. degree in Computer Engineering in University of Maryland, College Park, MD, USA, in 2018. His primary research topics include are in the area of approximate computing for low power and energy efficiency design, targeting on both computer architecture and machine learning algorithm. He also works on the hardwrae related security and trust problems.



Gang Qu (SM07) received his B.S. and M.S. degrees in mathematics from the University of Science and Technology of China, Hefei, China, in 1992 and 1994, respectively, and his Ph.D. degree in computer science from the University of California at Los Angeles, Los Angeles, CA, USA, in 2000. Upon graduation, he joined the University of Maryland, College Park, MD, USA, where he is currently a Professor with the Department of Electrical and Computer Engineering and the Institute for Systems Research. He is a member of the Maryland Cybersecurity Center and the Maryland Energy Research Center. He is the Director of Maryland Embedded Systems and Hardware Security Laboratory, College Park, and the Wireless Sensors Laboratory.

His primary research interests are in the area of embedded systems and very large scale integration (VLSI) computer aided design (CAD) with a focus on low power system design and hardware related security and trust. He studies optimization and combinatorial problems and applies his theoretical discovery to applications in VLSI CAD, wireless sensor network, bioinformatics, and cybersecurity. He has received many awards for his academic achievements, teaching, and service to the research community. He serves as an Associate Editor of the IEEE EMBEDDED SYSTEMS LETTERS, and the Integration, the VLSI Journal.

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.